AgenticCareerBoost

Agentic System Guide

Human-facing orientation manual

Dídac Llorens
github.com/DidacLL

April 2026 — v1.0

Contents

Chapter 1

What This Project Is

Key takeaway: AgenticCareerBoost is a public engineering campaign built as a path-based multiagent operating system inside a Git repository. The repository is not just where the project is documented; it is where the system runs.

The project exists to turn work into inspectable proof. Instead of describing a technical profile with adjectives, it records decisions, workflows, state, and formal outputs directly in versioned files. Humans can read the repository as a project manual. Agents can read it as an execution environment.

This guide is the human-readable entrypoint. The deeper technical evidence is in the bootstrap case study: `site/files/reports/s000-agentic-os-bootstrap.pdf`.

1.1 What it is not

- Not a startup product.
- Not a generic "AI wrapper" demo.
- Not a private automation setup hidden outside the repo.
- Not a replacement for human judgment.

Common pitfall: Do not mistake the repository structure for bureaucracy. The explicit files exist so humans and models can inspect the same truth with minimal ambiguity.

Chapter 2

Mission, Purpose, and Intended Audience

Key takeaway: The mission is to rebuild a public technical profile through visible, agentic engineering work and make that work readable by recruiters, peers, and future collaborators.

2.1 Mission and purpose

The canonical mission lives in `agents/rules/core/mission.md`. In practical terms, the system has three public jobs:

- convert scattered capability into clear engineering artifacts,
- keep repository, site, and social outputs coherent,
- prove the process, not only the result.

2.2 Intended audience

Curious readers
need a short explanation of what the repository is and why it exists.

Recruiters and hiring managers
need fast, credible evidence of systems thinking, execution discipline, and documentation depth.

Engineering peers
need the actual mechanism: roles, workflows, review loops, build paths, and state handling.

Operators and contributors
need to know how to request work, where to read next, and which files are authoritative.

2.3 How this guide fits the rest of the docs

- This guide explains the system in plain operational language.
- The README is the shorter repo entrypoint.
- The site is the public mirror.
- The S-000 case study is the detailed technical record.

Chapter 3

The System Mental Model

Key takeaway: The system has four layers: stable truth, workflow contracts, volatile state, and published outputs. Agents are routed by paths, not by a giant hidden prompt.

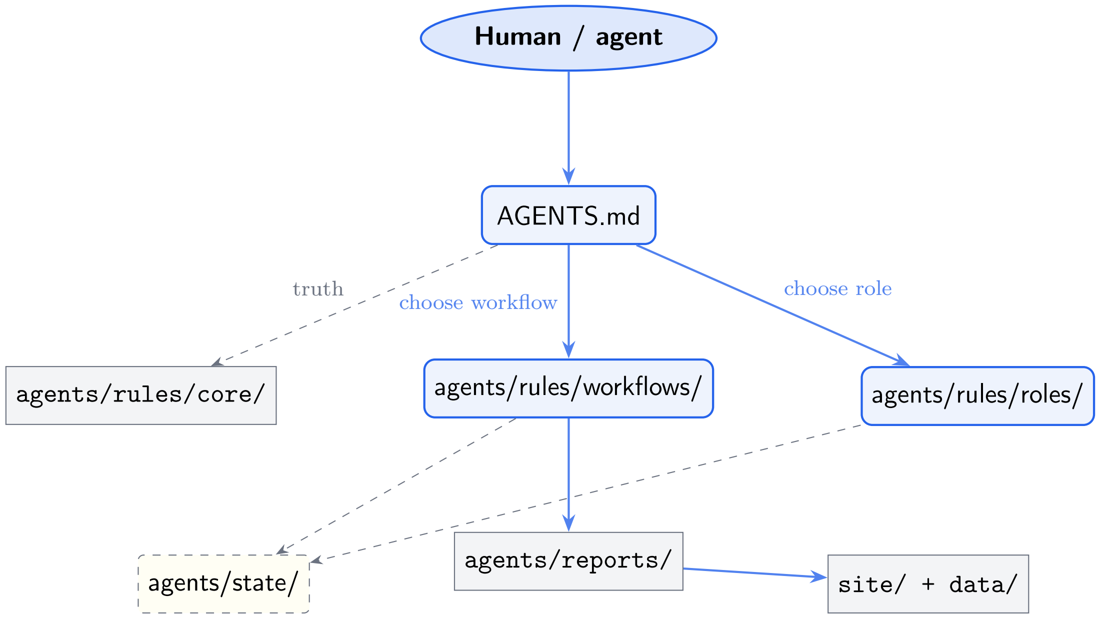

Figure 3.1: System mental model: entrypoint, contracts, state, and outputs.

3.1 Stable truth

Stable truth lives under `agents/rules/core/`. These files describe the mission, constraints, brand, and the truth hierarchy. They change rarely and win when a volatile file disagrees.

3.2 Workflow contracts

Workflow files define the allowed action modes: `chat`, `operate`, `review`, `hotfix`, `plan`, `sprint`, and `system-review`.

3.3 Volatile state

Volatile state lives under `agents/state/`. It holds the current status, backlog, active sprint, logs, and family memory. It is important, but it is not canonical truth when it conflicts with core rules.

3.4 Published outputs

Outputs live under `content/`, `site/`, and `data/`. The formal PDFs are part of that surface: they are meant to be read, cited, and inspected.

Chapter 4

How To Use The System As A Human

Key takeaway: Humans use the repository by choosing the right workflow, not by reading every file. Start from the question you are trying to answer or the change you are trying to make.

4.1 Use it as a curious reader

If you want to understand the project quickly:

1. Read the README.
2. Read this guide.
3. Open the S-000 case study only if you want deeper evidence.

4.2 Use it as an operator or collaborator

If you want to get work done:

1. Start at `AGENTS.md`.
2. Choose the workflow that matches the request.
3. Read only the referenced contracts and current state.
4. Keep changes inside declared scope and writes.

4.3 Use it as an agent

Agents should not improvise the filesystem. The correct path is: `AGENTS.md` → workflow → role or AutoAgent → state and outputs.

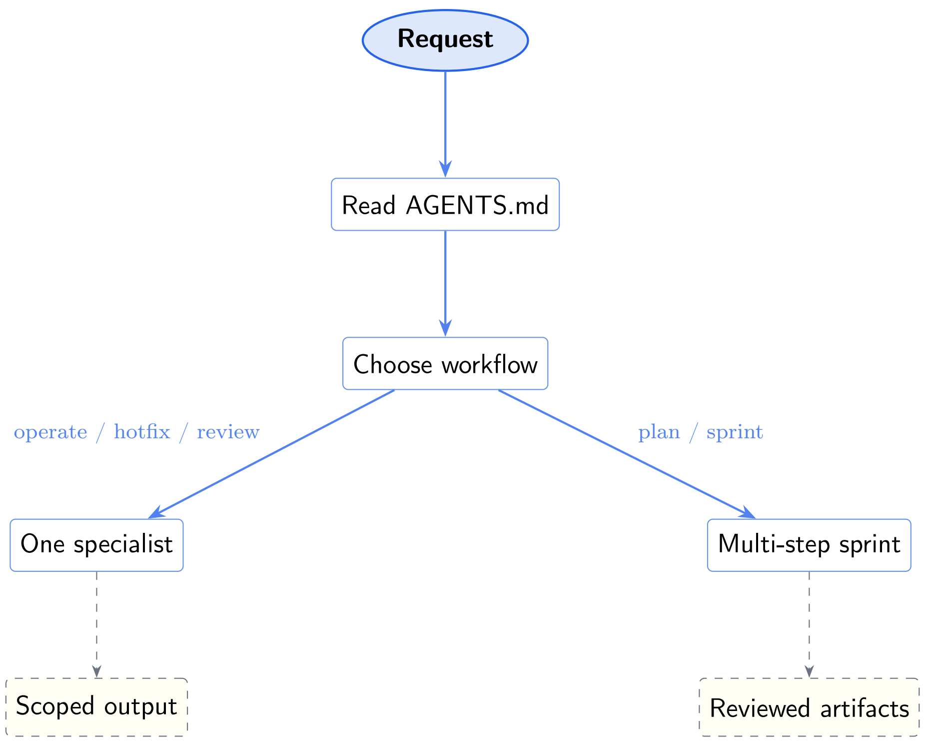

Figure 4.1: Human usage flow: choose the workflow before choosing the files.

Chapter 5

Actions And When To Use Them

Key takeaway: The action surface is intentionally small. Each workflow exists to solve a different coordination problem.

5.1 Action quick reference

Action	Use it when	Typical result
`chat`	You need discussion only	Answers, constraints, optional summary
`operate`	One bounded specialist should do one bounded job	One scoped artifact or correction
`review`	You need maintenance checks and consistency fixes	Scoped sync, lint, compile verification
`hotfix`	One small fix is clearly isolated	One commit and minimal backlog note
`plan`	A larger change needs a contract first	Active sprint definition
`sprint`	Planned multi-step work needs role coordination	Reviewed multi-artifact delivery
`system-review`	The architecture or rule system itself looks wrong	Issue report and proposed structural changes

5.2 Action selection flow

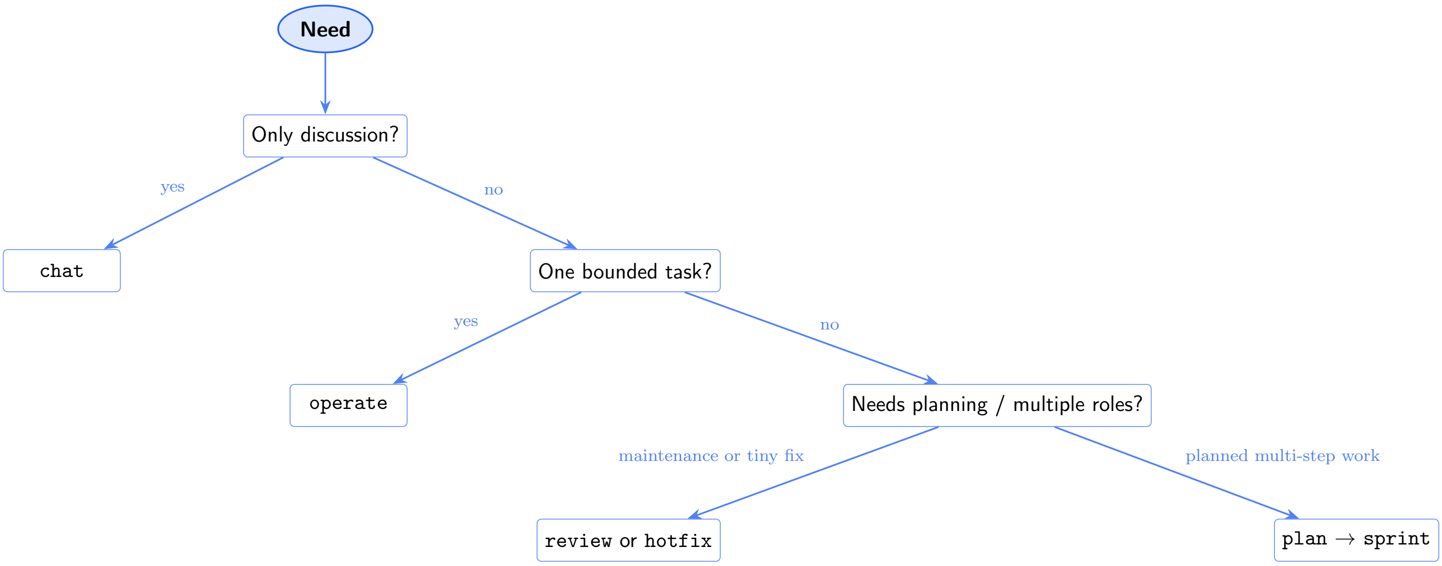

Figure 5.1: Choose the workflow by scope and coordination needs.

Chapter 6

Roles, Specialties, AutoAgents, And Memory

Key takeaway: Top-level roles stay small; specialties narrow execution; AutoAgents handle recurring routines; family memory stores durable heuristics, not truth.

6.1 Human-facing roles

- ORCHESTRATOR routes work and writes contracts.
- DEVELOPER implements bounded technical changes.
- PAIRCHECK reviews non-trivial outputs independently.
- CI/CD integrates and publishes approved work.
- DOCUMENTATION turns changes into readable engineering artifacts.
- COMMUNITYMANAGER adapts proof into public-facing messaging.

6.2 Specialties

Roles should not execute as generic experts. They run in specialty modes such as `Developer/latex`, `Documentation/guide`, or `CommunityManager/linkedin`. Specialty keeps the read scope small and the expected output explicit.

6.3 AutoAgents

The fixed routines currently live in `agents/rules/roles/autoagents.md`. They cover consistency, Markdown quality, LaTeX compilation, and social planning.

6.4 Family memory

Family memory lives in `agents/state/memory/`. It stores durable heuristics such as recurring build issues, accepted phrasing defaults, or source-quality notes. It must never replace canonical truth in `agents/rules/core/`.

Chapter 7

Repository Layout And Where To Start

Key takeaway: The repository is organized so that each question leads to one or two obvious next files.

7.1 Condensed layout

Path	Why you would open it
`AGENTS.md`	Start here if you are routing work
`agents/rules/core/`	Understand what is stable and non-negotiable
`agents/rules/workflows/`	Choose the correct action mode
`agents/rules/roles/`	Understand who owns the work
`agents/state/`	Read current status, backlog, and recent activity
`agents/reports/`	Read published formal documentation
`site/`	Inspect the public mirror source

7.2 Where a new reader should start

1. README for the shortest orientation.
2. This guide for the operating model.
3. S-000 for technical depth.
4. Specific workflows and roles only when you need to act.

Chapter 8

Typical Flows

Key takeaway: Most real usage falls into a few repeatable flows: read, operate, review, or plan.

8.1 Human orientation flow

README → this guide → S-000 case study.

8.2 Bounded maintenance flow

`review` → `ContentSync` first → Markdown or LaTeX specialist if needed → scoped fixes only.

8.3 Change delivery flow

`operate` for one bounded specialist task, or `plan` → `sprint` when the work spans multiple concerns and needs pair-check.

8.4 Documentation flow

Implementation change → documentation artifact → published PDF or repo/site update → optional public adaptation by CommunityManager.

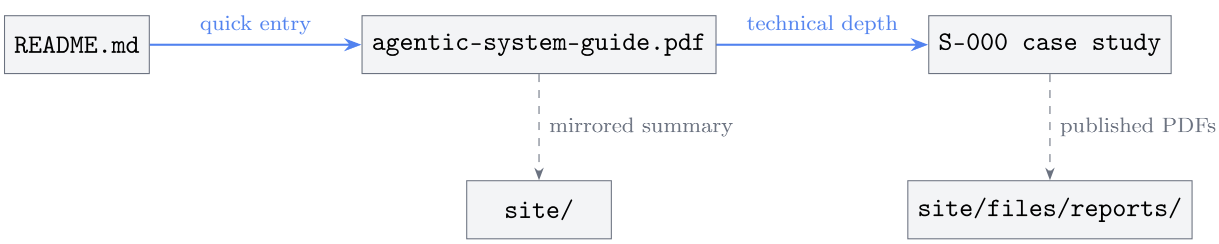

Figure 8.1: Documentation map: quick entrypoint, human guide, technical case study, and published outputs.

Chapter 9

Reading Path To Deeper Technical Material

> **Key takeaway:** Once the high-level model is clear, the deepest material is organized by question, not by chronology.

What is always true?
: Read `agents/rules/core/`.

What action should be used?
: Read `agents/rules/workflows/`.

Who owns each kind of work?
: Read `agents/rules/roles/`.

What changed recently?
: Read `agents/state/current.md` and the backlog.

How was the bootstrap system implemented?
: Read the S-000 case study.

How are formal PDFs built?
: Read `agents/reports/tex/README.md` and the local build scripts.

> **Common pitfall:** Do not start from logs unless you are debugging a specific failure. Logs are volatile evidence, not the best orientation path.

Appendix A

Quick Reference

Key takeaway: These are the highest-value paths for humans using the repository.

Path	Use it for
`README.md`	quickest human overview
`AGENTS.md`	routing entrypoint for agents and operators
`agents/rules/core/mission.md`	purpose and success criteria
`agents/rules/workflows/review.md`	maintenance review chain
`agents/rules/workflows/operate.md`	one bounded specialist task
`agents/rules/roles/autoagents.md`	recurring routines and triggers
`agents/state/current.md`	current status snapshot
`site/files/reports/agentic-system-guide.pdf`	published human-facing guide
`site/files/reports/s000-agentic-os-bootstrap.pdf`	published technical case study